

Instructing and Coordinating AI Agents: Key Properties and Techniques

System Prompts and Base Instructions

A **system prompt** (or base prompt) defines the AI's initial directives – essentially an invisible “job description” for the model ¹. This hidden instruction set establishes the agent's role, personality, and ground rules *before* any user interaction begins ². For example, consumer AI apps like ChatGPT or Claude include carefully crafted system prompts that control the assistant's **tone, refusal style, safety boundaries**, and even response formatting ². When using an API to build a custom agent, developers must provide their own system prompt; otherwise the raw model will lack a defined persona or consistent style, and may respond unpredictably without the desired rules or formatting ³. In other words, the system prompt is the foundation that keeps AI behavior **consistent, safe, and on-message** across sessions.

A good system prompt typically outlines *who* the AI is, *how* it should behave, and *what* it should or shouldn't do. This might include the AI's role or identity (e.g. “*You are a friendly customer service assistant for an e-commerce company*”), its communication style (e.g. *friendly and professional*), and explicit directives or restrictions (for instance, instructions to refuse certain requests or avoid specific content). Notably, system-level instructions have higher authority than user prompts in most frameworks ⁴. That means if the user tries to override the AI's persona or rules, the underlying system prompt can trump those attempts (within the model's capabilities) to enforce the desired behavior. This hierarchy helps protect against malicious *prompt injection* and keeps the AI aligned with its intended role ⁵.

Example – Persona Definition via System Prompt: Consider a YAML snippet defining a weather assistant agent's system prompt:

```
system_prompts:
  weather_expert: |
    You are Enrico, a friendly and helpful weather assistant.

    Your responses will be converted to audio, so avoid using special characters or
    symbols.

    Keep your tone friendly and conversational, and always respond in English.
    - Use the get_weather tool ONLY when the user specifically asks about the
    weather or atmospheric conditions.
    - If the user asks anything else, answer normally as a weather expert
    without using any tools.
    - Your main areas of expertise are current weather conditions, forecasts,
    and climate patterns.
```

In this example, the system prompt assigns the AI a persona ("**Enrico**, a friendly and helpful weather assistant") and includes special instructions because the answers will be spoken aloud (avoid unusual symbols). It also sets the **tone** ("friendly and conversational") and language (English) for all replies, and outlines tool-usage rules ⁶ ⁷. This kind of structured prompt ensures the agent knows *how* to act (personality and style) and *what procedures* to follow when handling queries (e.g. when to call a weather API).

Persona, Tone, and Communication Style

Persona refers to the identity or role the AI is instructed to adopt. Defining a persona means telling the model *who it is* in the context of the conversation – for example, a wise professor, a playful travel guide, or a corporate customer support agent. Along with the persona, designers also specify the AI's **tone of voice** and general communication style so that its responses feel consistent and aligned with that character or brand. In prompt engineering terms, persona is sometimes called the AI's "role" or "voice." It strongly influences how the AI interprets queries and frames its answers ⁸.

When configuring an AI agent's persona in practice, several facets come into play:

- **Identity and Role:** Who is the AI pretending to be? (e.g. "*You are a math tutor here to help students with homework*" ⁹ or "*You are an AI fashion stylist assisting a customer.*") This sets context about the agent's knowledge domain and perspective. Some systems let you name the agent and choose a role type (for instance, *acting as the user's clone, as a personal assistant, as a fictional character, or as a brand persona*) ¹⁰. The chosen role can affect whether the AI speaks in first person as if it *were* the user, or in third person as a separate helper, etc. ¹⁰.
- **Tone and Style:** The manner of language the AI should use – for example, **polite and professional, casual and friendly, enthusiastic and upbeat**, or perhaps **witty and humorous**. Tone directives can be explicit (e.g. "*Respond in a warm, empathetic tone*") or implicit in the persona description (a *cheerful tutor* will likely use simple, encouraging language). Many platforms allow choosing predefined tone settings; for instance, one support platform offers "Professional (polite and direct), Informal (casual and friendly), or Enthusiastic (upbeat and friendly)" as selectable styles ¹¹. Tone is essentially the *mood* or *voice* of the AI's replies, and is critical for making interactions feel natural. (Notably, setting a persona often *implies* a certain tone – a pirate persona might speak in "Arr!" slang – but tone can also be tuned independently ¹².)
- **Vocabulary and Formality:** Related to tone, one can specify if the AI should use simple language or technical jargon, contractions or formal grammar, emojis or no emojis, etc. For example, an AI geared toward children should use simpler words and a gentle tone, whereas a legal advisor AI might use precise, formal language. These style guidelines are often part of the persona's definition.
- **Domain Knowledge or Expertise:** Often the persona includes what subjects the AI specializes in or its background. For instance, "*You are a financial advisor with expertise in retirement planning*". This helps the model focus on relevant information and phrase answers appropriately (perhaps citing financial terms in that case). It also informs the **behavior** – e.g. a medical AI should be cautious and factual, while a fiction storytelling AI can be more imaginative.

- **Reply Length and Format:** Another aspect of style is how verbose the agent should be. Some configurations let you set a preferred answer length (for example, *brief (1-2 sentences)* vs. *detailed (4-5 sentences)*). In one example, an enterprise chatbot persona was defined with a “*messaging reply length: short (≈50 words) or long (≈80 words)*” to keep answers consistently concise or more explanatory as needed ¹³ ¹¹. You might also specify formatting preferences (like using bullet points, or outputting JSON, etc., as part of the persona’s communication style).

Overall, persona and tone settings ensure the AI’s responses have a **coherent character and style**. Consistency is key – users should feel like they are interacting with the *same personality* over time. Many modern AI frameworks provide a way to encapsulate these settings. For example, Zendesk’s AI agent persona configuration consists of a short **profile description** (to give context about the business domain), a chosen **tone of voice** (e.g. professional vs. informal), and a target **reply length**, which together give the AI a consistent voice aligned with the brand ¹⁴. Other systems (like Personal AI) allow setting custom **traits** and even different style rules per communication channel ¹⁵ ¹⁶. In all cases, the goal is to finely tune *how* the AI speaks and behaves, independent of *what task* it’s doing.

Adapting to Different Communication Mediums

AI agents designed for human interaction may communicate via **text**, **voice**, or even **video/avatar** interfaces. The underlying language model might be the same, but we often adjust the prompts and outputs for each medium to ensure effective communication:

- **Text-Based Interaction:** This is the simplest medium – the AI outputs written text in a chat, email, or message. Here the focus is on clarity and readability. In text, we can easily use formatting (like Markdown headings, bullet points, or code blocks) if appropriate. The system prompt might specify such formatting conventions or encourage certain structures (for example, “*always provide a brief summary, followed by bullet-point steps*” as a formatting rule ¹⁷). Tone and persona are conveyed purely through word choice and style. A text-based AI can often handle longer, more detailed responses since the user can scroll and reread if needed.
- **Voice (Spoken) Interaction:** For voice agents (e.g. AI assistants on phone, smart speakers, or any scenario where text-to-speech is used), additional considerations apply. **Brevity and clarity** become more important because the user has to listen to responses. Prompts might instruct the AI to keep sentences short and use spoken-friendly language (avoiding very long or complex sentences that are hard to follow by ear). It’s also common to forbid or limit unusual characters, emojis, markdown, etc., because those don’t translate well to speech. For example, a system prompt for a voice-based assistant might say: “*Your responses will be converted to audio, so do not include any special symbols or formatting that wouldn’t make sense when spoken*” ¹⁸. The tone for voice output should be explicitly friendly and conversational if you want a natural speech cadence ¹⁸. Additionally, the choice of the **synthetic voice** (TTS engine) is part of the persona – you might pick a calm female voice or an enthusiastic male voice, etc., to match the intended personality. Advanced setups can use SSML (Speech Synthesis Markup Language) tags in the output to control prosody, emphasis, pauses, and pronunciation in the spoken rendition. In sum, designing for voice means ensuring the AI’s generated text *sounds* good when read aloud.
- **Video / Avatar Interaction:** When an AI is represented by an avatar or digital human in video, we have all the concerns of voice (since the avatar will speak the AI’s responses) plus visual presentation.

The persona here extends to the avatar's appearance, facial expressions, and body language. Systems like AI avatar generators allow customization of the avatar's **look, voice, language**, and even **gestures** to align with the desired personality ¹⁹. For example, a cheerful customer service avatar might have a friendly face and warm expressions, with a gentle tone of voice and welcoming body language. The AI's text output might be augmented by the avatar automatically with appropriate non-verbal cues (smiles, nods) based on the content and sentiment of the response. Prompt-wise, one might remind the agent of the visual context – e.g. if an avatar can display emotions, the AI could include stage-directions like "[smile]" or the system could infer them. However, often the avatar system handles expressions, so the AI mainly ensures the *content and tone* are in line. Also, when delivering via video, it's crucial that responses stay **on-brand and engaging** – a monotone or inconsistent persona will be very noticeable when embodied in an interactive video character. Thus, developers will tune both the underlying AI (through persona and style prompts) **and** the avatar parameters (voice model, appearance, animation style) to create a cohesive experience ²⁰ ¹⁹. For instance, HeyGen (an AI avatar platform) notes that you can control an avatar's aesthetic and speaking style to range from "professional and polished" to "fun and quirky," ensuring the visual persona matches the communication style of the AI's responses ²⁰.

Regardless of medium, the core idea is to **maintain the AI's personality and clarity**. A well-designed AI will adapt its manner of expression to the channel: text agents might provide hyperlinks or screenshots, voice agents might spell out complicated names or numbers and use auditory cues ("First, ... Next, ..."), and avatar agents will combine speech with visual engagement. The underlying prompt and orchestration should reflect these needs. Often, separate prompt templates or **channel-specific directives** are used for different output channels ¹⁶. For example, you might have one version of the system instructions for chat, and another slightly tweaked for voice (shorter answers, no markdown), each ensuring the AI remains effective in that medium.

Memory Management and Persistence

One limitation of many AI systems is a lack of long-term memory – they treat each session or query in isolation unless explicitly provided context. Effective agent design therefore must handle **memory** in two senses: **short-term conversational memory** and **long-term/persistent memory**.

- **Short-Term Memory (Context):** This refers to the immediate context window of the model – essentially what the AI can remember from the ongoing dialogue. In a chat scenario, short-term memory is usually handled by including recent conversation history in the prompt for each new turn. For instance, when you have a back-and-forth with ChatGPT, the system is resending a certain number of previous messages along with your latest query so that the AI has context to respond coherently. Frameworks like LangChain or LlamaIndex provide **memory modules** that automate this process: they maintain a buffer or summary of the recent chat and prepend it to each prompt ²¹ ²². The system prompt can also remind the AI to *use conversation context* when appropriate (e.g. "Use the user's prior messages as context; do not ask them to repeat information already given"). However, short-term memory is limited by the model's token capacity, so designers often have to truncate or summarize older parts of the conversation when it gets too long. A common technique is **summarization memory** – earlier dialogue is distilled into a concise summary that stays in the prompt, freeing space while retaining important facts or the user's goals.

- **Long-Term Memory / Persistence:** By default, most AI agents **forget everything** once a session ends or the context limit is hit. This leads to the well-known “**amnesia problem**” where, for example, a chatbot doesn’t recall you talked to it yesterday about your preferences ²³. To make AI interactions more seamless and personalized, developers implement persistence layers. This can involve storing information from past interactions in a database or **vector memory** so the agent can retrieve it in future conversations. Common approaches include: **Vector Databases** (embedding key facts or conversation snippets as high-dimensional vectors that can be similarity-searched later), **knowledge graphs** (storing structured facts the AI has learned about the user or environment), or fine-tuning/updating the model (less common in real-time, due to cost). For instance, important details like a user’s name, their preferences, or unresolved issues might be saved after a chat. Next time the user returns, the system can pull those facts and feed them into the prompt (e.g. “*The user’s name is Alex and their last query was about credit card rewards*”). Recent research and tools have been focusing on this; an AI agent can be equipped with “human-like” memory by leveraging vector stores and knowledge bases to recall past events ²⁴. There are already libraries (e.g. LangChain’s **ConversationBufferMemory** with persistence, or LangGraph’s memory in LangChain agents ²⁵) that let you attach long-term memory to agents, so that *session-to-session persistence* is possible.

Effective memory management often uses a combination of strategies: **ephemeral memory** (the immediate dialogue), **working memory summaries**, and **long-term storage** for important facts or frequently needed data. Developers must also design retrieval logic – the agent needs to know when to fetch something from long-term memory (perhaps triggered by a user mentioning a known topic, or just retrieving a profile every time a session starts). Done well, this makes the AI feel far more coherent and personalized over time, as it doesn’t constantly reset. The user no longer has to repeat known information, and the AI can **learn from past interactions** (within the bounds of what it’s allowed to store, respecting privacy and safety).

From an orchestration perspective, memory can be treated as a tool or resource. For example, an agent might have access to a function like `retrieve_fact(query)` to search its knowledge base, or a function `save_note(note)` to store new information. Advanced agent frameworks treat memory as a first-class citizen – one proposal even talks about a “*Memory OS*” that uniformly manages different kinds of memory (conversation history, knowledge, vector memories) for AI systems ²⁶. While that’s an active research area, the practical takeaway is that instructing an AI agent involves not just guiding its behavior, but also **feeding it the right memories** at the right time to ground that behavior.

Safety Filters and Alignment

Ensuring the AI communicates safely and within acceptable boundaries is another crucial aspect of AI agent orchestration. This is handled through a combination of **prompt-level instructions (guardrails)** and **external moderation systems**:

- **Prompt-Based Safety Instructions:** The system prompt usually includes explicit restrictions to prevent unwanted behavior. For example, it might state rules like “*If the user requests disallowed content (self-harm, hate, violence etc.), the assistant must refuse politely*” or “*Never provide medical or legal advice beyond general information. If pressed, suggest consulting a professional.*” These act as ethical and compliance guardrails. In fact, the hidden prompts in services like ChatGPT contain detailed policies about what the AI should refuse or avoid. A system prompt for a coding assistant, for instance, might include: “*Never generate or explain malware or exploits; if asked to do so, respond*”

with a brief refusal.”²⁷ . By baking these rules into the AI’s persona and behavior, the agent is more likely to handle problematic queries safely. Other common safe completions include apologizing and saying one cannot comply, or redirecting the user to appropriate resources (especially for things like mental health crises).

- **Model Alignment and Training:** It’s worth noting that many models (especially OpenAI and Anthropic models) come **pre-trained with alignment** techniques (like reinforcement learning from human feedback) that make them follow general ethical guidelines and refuse disallowed content. Those higher-level training-based guardrails operate even beyond your prompt – for example, even if you didn’t explicitly tell it to avoid violent instructions, the model might still refuse due to its built-in alignment. That said, relying on training alone isn’t enough, so developers reinforce with prompt instructions as above.
- **External Content Filters:** In addition to instructing the AI, most production systems put an **automated moderation layer** around the AI’s inputs and outputs²⁸ ²⁹ . This typically involves intercepting the user’s query before it goes to the AI (to block or cleanse blatantly disallowed requests), and/or scanning the AI’s answer before it reaches the user (to ensure no policy violations slip through). These filters can use simple keyword matching (e.g. flag any credit card numbers to prevent PII leakage) and machine learning classifiers for things like hate speech or sexual content²⁹ . Some setups use a secondary AI model as a “safety inspector” – for instance, a smaller language model that reads the draft output and checks for red flags or forbidden content categories³⁰ . If something unsafe is detected, the system might replace the response with a generic refusal or a sanitized version. External guardrails are important because they don’t rely on the primary model’s cooperation; they act as a failsafe. Common categories to filter include: harassment/hate, violence, sexual content, self-harm, personal data leakage, malware or criminal advice, and model self-revelation (prompts trying to get the AI to divulge its hidden instructions)³¹ . The strictness of filters can often be tuned to the use case (a kids’ app will be extremely strict, whereas a research tool might allow more leeway except for truly harmful content)³² .
- **Iterative Feedback and Policies:** In orchestrating AI communication, designers also continuously refine the safety rules. Many organizations maintain a **policy document** for their AI’s behavior and update the system prompt or filtering rules as new failure modes are discovered. There are also emerging tools (open-source “AI guardrails” libraries) that provide templates for common safety rules which you can apply to your agent. Ultimately, safety is about both **what the AI says and how it reacts**. Tone is part of this too – for example, if refusing, the AI should do so in a manner that is firm but not disrespectful or overly detailed (to avoid giving hints about how to bypass the refusal, etc.). This refusal style can be explicitly set in the system prompt as well² .

In summary, coordinating an AI agent includes embedding robust safety constraints through prompts (to align the AI’s behavior) and using external moderation layers as a safety net²⁸ ²⁹ . This two-layer approach helps ensure the AI remains a **helpful and harmless** assistant in all its interactions.

Multi-Agent Orchestration and Simulations

So far we’ve discussed properties of a *single* AI agent interacting with humans. In some scenarios, you have **multiple AI agents** working together or conversing among themselves (and possibly with humans) – for example, a team of specialized bots solving parts of a problem, or a simulation with many AI characters.

Multi-agent orchestration is the process of coordinating these agents so that they behave in a coherent, goal-directed way ³³ .

In a multi-agent system, each agent typically has its own persona, role, and specialty. For instance, in an automated customer service workflow, you might have one agent that's an Order Assistant (handles order status queries), another that's a Tech Support Agent (handles product troubleshooting), and a third that is an Escalation Bot (decides if a human manager is needed). The challenge is to have them communicate and cooperate effectively, rather than working at cross purposes. Often a dedicated *orchestrator* component (which could be another AI agent or a rule-based system) manages this interaction ³⁴ ³⁵ . The orchestrator's job is to route tasks to the right agent, ensure they share necessary context, and combine their outputs towards the final goal ³⁶ ³⁴ . IBM describes this as a "digital symphony" – each agent is like a musician with a specific part, and the orchestrator is the conductor ensuring they stay in harmony ³⁷ .

Key considerations in multi-agent setups include:

- **Communication Protocol:** Agents might communicate via a messaging protocol (passing text messages back and forth that the orchestrator or a channel mediates). A common approach is to have agents produce messages like, "*Agent A -> Agent B: [content]*", which the system then feeds into Agent B's prompt, simulating a conversation. The system may need to prevent infinite loops or chaotic chatter. Clear turn-taking or a defined sequence (sometimes even a *meeting agenda*) can be imposed. In simpler cases, agents don't talk directly but work on parts of a shared task and send results to a central system.
- **Shared Memory/Context:** To collaborate, agents often need a shared knowledge base or memory. Multi-agent orchestration frameworks provide a **common context** or blackboard that all agents can read and contribute to ³⁸ ³⁹ . For example, if Agent A finds some data, it could post it to the shared context so Agent B can use it. Alternatively, agents might call each other's APIs or functions rather than free-form chatting. In complex simulations (like multi-agent role-play environments or games), a simulated **environment state** acts as shared context – each agent observes the environment (and possibly the actions of others) and then decides what to do. In such cases, designing *what each agent perceives and remembers* is crucial, so you might limit agents to partial knowledge or give them each a memory module similar to single-agent memory, but also allow some global facts accessible to all.
- **Coordination Strategies:** There are various architectures: **centralized orchestration** (one top-level agent or program directs all others) ⁴⁰ , **hierarchical** (layers of agents where a manager assigns tasks to sub-agents) ⁴¹ , or **decentralized/emergent** (agents have lots of autonomy and only light rules of engagement, potentially self-organizing their collaboration) ⁴² . For business workflows, a centralized or hierarchical approach is common for reliability. For research simulations, emergent orchestration is interesting – for example, the recent "*Generative Agents*" research by Stanford (2023) showed 25 AI characters living in a sandbox town, each with their own memories and routines, interacting autonomously with only minimal high-level prompting. They ended up forming opinions and having social interactions that appeared quite human-like, all through multi-agent emergent behavior ⁴³ . In such a simulation, consistency of persona (each agent had a backstory and character profile), and robust memory (each agent logged observations and recalled relevant ones) were key to making their interactions believable.

- **Tools and Environment:** Multi-agent systems often involve agents performing actions, not just chatting. This could mean calling external APIs, running code, or manipulating some world state. An orchestrator might facilitate this by giving certain agents permission to use certain tools. For example, one agent might be designated to execute code, another to fetch information, and so on, and others must request that agent's help when needed. Establishing these capabilities and protocols is part of the "coordination properties" of the system prompt at a meta-level.
- **Human Interaction and Mediation:** In many cases, multi-agent setups still interact with humans (either as the end-user or as oversight). You may have a scenario where two AI agents discuss a plan (say, a doctor AI and a patient AI simulating a consultation) and a human overseer monitors or can step in. Designing how the AI agents communicate their reasoning or ask for help is another piece. Often agents are instructed to *not* reveal their internal reasoning to the user unless formatted in a specific way, to avoid confusion ⁴⁴. For multi-agent, this might extend to not revealing internal agent-to-agent messages, etc.

In practice, coordinating multiple agents requires everything you need for one agent **plus** additional layers of organization. Each agent will have its own prompt (persona, knowledge, style) and there is usually a top-level script or system prompt that outlines the overall **teamwork plan** – for instance, assigning roles and explaining how they should collaborate. A simple example: *"Agent1 is the Planner and will break down the problem. Agent2 is the Solver who will carry out calculations. Agent1 and Agent2, you will communicate in turns: Planner, then Solver, etc., until the problem is solved."* This would be a high-level instruction given to both in their context, so they know the expected protocol. More sophisticated frameworks exist (e.g. Microsoft's *Autogen*, OpenAI's *Function Calling* used to chain tool-using sub-agents, or libraries like *LangChain* that support multi-agent dialogues). These frameworks handle message passing and can enforce an order of speaking or a fallback to a human if the agents get stuck.

Important notes for simulations: When many AI agents interact, unpredictability can increase. It's important to still enforce safety – one rogue agent shouldn't be allowed to produce unsafe content unchecked. Often the same safety filters apply to agents' communications as to human-facing messages. Also, performance and cost can become an issue with multiple agents (since each agent call is like another large model query). Therefore, some orchestrations use a mix of "big" and "small" models (specialized smaller models for certain tasks, larger ones for complex reasoning) to optimize.

Finally, evaluating multi-agent systems is an active area of research: metrics for how well they cooperate, if they stay in character, and if the overall system achieves its goals are still being developed ⁴⁵. But from a high-level perspective, the **properties and data used in AI agent coordination** – whether single or multi-agent – boil down to *defining clear roles/personas, setting the tone and interaction style, providing memory/context, ensuring safety, and establishing protocols for communication*. By carefully designing these elements (often via structured prompts or configuration schemas), we can orchestrate AI agents that communicate effectively with humans *and* with each other across text, voice, and video mediums.

References and Sources

- OpenAI and Anthropic system prompt practices for tone, style, and safety ²
- Example of a structured system prompt defining an AI's persona, tone, and tool use ⁶ ⁷
- Zendesk AI persona settings (business profile, tone, reply length) ¹⁴ ¹¹
- Google's prompt design components: persona, tone, constraints, format, etc. ⁸ ¹²

- HeyGen avatar customization for voice/video (look, voice, gestures, style) 20 19
- Multi-agent orchestration concepts and definitions 33 37
- Talkdesk overview of single-agent vs multi-agent coordination 46
- Memory and persistence in AI agents (the “amnesia problem” and long-term memory via vector DB) 23 24
- External safety layer techniques for LLM moderation 28 29

1 2 3 4 5 17 27 44 **Guide to Writing System Prompts: The Hidden Force Behind Every AI Interaction**

<https://saharaai.com/blog/writing-ai-system-prompts>

6 7 18 21 22 **FastRTC Voice AI Agent. With the launch of the open-source... | by Enrico Nello | The Deep Hub | Medium**

<https://medium.com/thedeephub/fastrtc-voice-ai-agent-534aa8dec899>

8 9 12 **Overview of prompting strategies | Generative AI on Vertex AI | Google Cloud Documentation**

<https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/prompt-design-strategies>

10 15 16 **Personas: Customizing your Personas**

<https://www.personal.ai/pai-academy/personas-customizing-your-personas>

11 13 14 **Configuring an AI agent’s persona to add personality to AI-generated responses – Zendesk help**

<https://support.zendesk.com/hc/en-us/articles/8753435048474-Configuring-an-AI-agent-s-persona-to-add-personality-to-AI-generated-responses>

19 20 **Interactive AI Avatars for Smart Engagement | HeyGen**

<https://www.heygen.com/interactive-avatar>

23 24 **AI Agents That Remember: Building Long-Term Memory Systems | by Micheal Lanham | Medium**

<https://medium.com/@Micheal-Lanham/ai-agents-that-remember-building-long-term-memory-systems-dff6e6b7cdae>

25 **Add memory - GitHub Pages**

<https://langchain-ai.github.io/langgraph/how-tos/memory/add-memory/>

26 **\titlefontMemOS: A Memory OS for AI System - arXiv**

<https://arxiv.org/html/2507.03724v2>

28 29 30 31 32 **Keeping LLMs in Check: A Practical Guide to External Safety Layers - RisingStack Engineering**

<https://blog.risingstack.com/llm-safety-layers/>

33 37 **What is AI Agent Orchestration? | IBM**

<https://www.ibm.com/think/topics/ai-agent-orchestration>

34 35 36 38 39 40 41 42 46 **What is Multi-Agent Orchestration? An Overview | Talkdesk**

<https://www.talkdesk.com/blog/multi-agent-orchestration/>

43 **Generative Agents: Interactive Simulacra of Human Behavior - arXiv**

<https://arxiv.org/abs/2304.03442>

45 **Designing Multi-Agent Intelligence - Microsoft for Developers**

<https://developer.microsoft.com/blog/designing-multi-agent-intelligence>